

High-Density Object Segmentation Baseline, Deep Learning & Hybrid Study

Pranay Sarkar

May 2026

Classical baselines · Fine-tuned YOLOv8s-seg · Density-aware hybrid All results from a
single evaluation run — 2026-09-22, seed 1337, 50 held-out test images

Abstract

Segmenting and counting individual object instances in densely crowded images remains a difficult problem in computer vision. This study compares two classical methods — Watershed and KMeans colour clustering — against a fine-tuned YOLOv8s instance-segmentation model and a density-aware hybrid, on the subset of COCO val2017 containing 5–50 non-crowd objects per image. All four methods are evaluated in a *single* run on the same 50 held-out images under identical, class-agnostic metric definitions. Watershed systematically under-counts (4.2 predicted against 12.6 true objects, MAE 8.66, 28% count accuracy) and KMeans catastrophically over-counts (127.5 regions, MAE 114.86, 0% accuracy) — both for structural reasons rather than for want of tuning. Fine-tuning YOLOv8s-seg for 10 epochs reaches 52% accuracy at 0.429 mean IoU. The hybrid, which detects crowding from Canny edge density before relaxing the detector and cross-checking against Watershed, reaches 64% accuracy (12 points above the detector alone) and reduces counting MAE by 23%. An ablation attributes 46 of the 49 objects of recovered counting error to the relaxed detector pass and only 3 to the Watershed cross-check, so the method is more accurately characterised as density-aware two-pass detection with a marginal classical correction than as an equal coupling of the two paradigms. We also show that mean IoU alone is insufficient on this task: KMeans attains 0.252 IoU while getting the object count right on none of the 50 images, because it emits roughly ten times more regions than there are objects.

Contents

1	Introduction	3
2	Literature Review	3
2.1	Classical Image Segmentation	3
2.2	Watershed Algorithm	3
2.3	COCO Dataset	4
2.4	YOLO Family and YOLOv8	4
2.5	Instance Segmentation and Mask R-CNN	4
2.6	Benchmarking with COCO mAP	4
2.7	Gap Addressed by This Project	4
3	Dataset & Exploratory Data Analysis	5
3.1	Dataset Overview	5
3.2	Object Density Distribution	5
3.3	Category Distribution	5
3.4	Key Challenges	6
4	Phase 1 Methodology — Classical Baselines	6
4.1	Watershed Segmentation	6
4.2	KMeans Colour Segmentation	7
5	Phase 2 Methodology — Deep Learning with YOLOv8	7
5.1	YOLOv8 Architecture	7
5.2	Instance Mask Generation	8
5.3	Loss Functions	8

5.4	Fine-tuning Strategy	8
5.5	mAP Evaluation Metric	9
6	Phase 3 Methodology — Hybrid System	9
6.1	Architecture Overview	9
6.2	Training Configuration	10
6.3	Key Innovation: Conditional Coupling	10
7	Results	10
7.1	Metric definitions	11
7.2	The classical baselines fail structurally	11
7.3	Mean IoU alone is not a sufficient metric	12
7.4	Latency	14
8	Ablation: which mechanism produces the gain?	14
8.1	What this ablation does not establish	15
9	Discussion	15
9.1	Why the detector handles crowding better	15
9.2	Why relaxing the detector helps in crowds	15
9.3	Threats to validity	16
10	Conclusion	16

1 Introduction

Instance segmentation — the task of detecting every object in an image and delineating its precise pixel-level boundary — is one of the most challenging problems in computer vision. The difficulty increases dramatically in *high-density* scenes where dozens of objects overlap, occlude one another, and vary widely in scale.

Dense object segmentation has numerous real-world applications:

- **Retail shelf analysis:** automatically counting products on crowded supermarket shelves for inventory management.
- **Crowd analysis:** estimating crowd density and tracking individuals in surveillance footage.
- **Biological cell imaging:** counting and segmenting cells in microscopy images where cells frequently touch or overlap.

The fundamental obstacles in dense segmentation are twofold. First, **occlusion** causes object boundaries to be partially or fully hidden, making it impossible to recover shape information from pixel intensities alone. Second, **scale variation** means that objects in the same image can range from a few pixels to thousands, rendering fixed-scale processing pipelines inadequate.

The objective of this project is to establish a rigorous baseline by applying classical segmentation methods to a curated dense subset of the COCO benchmark, quantifying their limitations, and thereby motivating the adoption of deep-learning methods in Phase 2.

2 Literature Review

2.1 Classical Image Segmentation

Graph-based segmentation methods treat each pixel as a node in a graph and define edge weights based on visual similarity. Felzenszwalb and Huttenlocher [1] proposed an efficient algorithm with $O(n \log n)$ complexity that merges regions greedily when the inter-region difference is smaller than the intra-region variation. While elegant, graph-based methods struggle in dense scenes because the visual similarity between adjacent objects of the same category produces weak edge weights, leading to under-segmentation.

2.2 Watershed Algorithm

The Watershed algorithm, formalised by Vincent and Soille [2], treats the greyscale image as a topographic surface. “Water” fills local minima (catchment basins) and rises until basins merge; the lines where they meet form segmentation boundaries. Pre-processing with a distance transform converts the binary foreground into a height map so that object centres become peaks and touching boundaries become valleys. However, when objects are in physical contact — as is common in crowded images — the valley between them vanishes, causing over-merging.

2.3 COCO Dataset

The Microsoft COCO (Common Objects in Context) dataset [3] is the de facto standard benchmark for object detection and instance segmentation. It contains over 200,000 labelled images across 80 object categories, with polygon-level annotations for each instance. The validation split (val2017) used in this study contains 5,000 images and 36,335 annotations. COCO’s emphasis on “objects in context” — natural scenes with multiple, often overlapping objects — makes it ideal for studying dense segmentation.

2.4 YOLO Family and YOLOv8

The You Only Look Once (YOLO) family, introduced by Redmon et al. [4], revolutionised real-time object detection by framing detection as a single regression problem over a spatial grid, avoiding the two-stage region proposal bottleneck. Each grid cell predicts bounding boxes, confidence scores, and class probabilities in a single forward pass. YOLOv8 [5], developed by Ultralytics, extends the YOLO paradigm with an anchor-free detection head, improved CSPDarknet backbone, and a prototype-based instance segmentation head. It achieves state-of-the-art speed-accuracy trade-offs on COCO, surpassing prior YOLO versions and closing the gap with two-stage methods on the mAP50-95 metric.

2.5 Instance Segmentation and Mask R-CNN

He et al. [6] introduced Mask R-CNN, a two-stage detector that extends Faster R-CNN with a parallel branch for predicting binary instance masks. The key insight is that instance masks can be decoded from region-of-interest (RoI) aligned features, decoupling class prediction from mask generation. Two-stage detectors generally achieve higher mask quality than single-stage methods (e.g. YOLO) at the cost of higher inference latency, making single-stage approaches preferable for real-time dense-scene applications.

2.6 Benchmarking with COCO mAP

The COCO benchmark evaluates instance segmentation using mean Average Precision (mAP), computed by averaging the AP over all 80 categories and over IoU thresholds from 0.50 to 0.95 in steps of 0.05 (mAP50-95). This metric is stricter than the PASCAL VOC mAP50 standard and rewards precise localisation. For dense scenes, mAP50 is often reported separately as a measure of detection recall at relaxed localisation.

2.7 Gap Addressed by This Project

While individual classical methods have been evaluated on simple benchmarks, no single study systematically compares Watershed and colour clustering on a density-stratified subset of a modern benchmark like COCO, and then directly compares them to a fine-tuned YOLOv8 model on the same images. This project fills that gap by providing a controlled ablation study that quantifies exactly *where* and *why* classical methods fail and by how much deep learning improves over them on the dense subset.

3 Dataset & Exploratory Data Analysis

3.1 Dataset Overview

Table 1: COCO val2017 summary statistics.

Metric	Value
Total images	5,000
Total annotations (non-crowd)	36,335
Unique categories	80
Avg objects / image	7.27
Min objects / image	0
Max objects / image	62
Dense images (5–50 obj.)	2,463

3.2 Object Density Distribution

Figure 1 shows the distribution of object counts per image. The vast majority of images contain fewer than 15 objects, with a sharp drop in frequency beyond 20. Only 60 images (1.2%) fall in the “very high” band (30–50 objects).

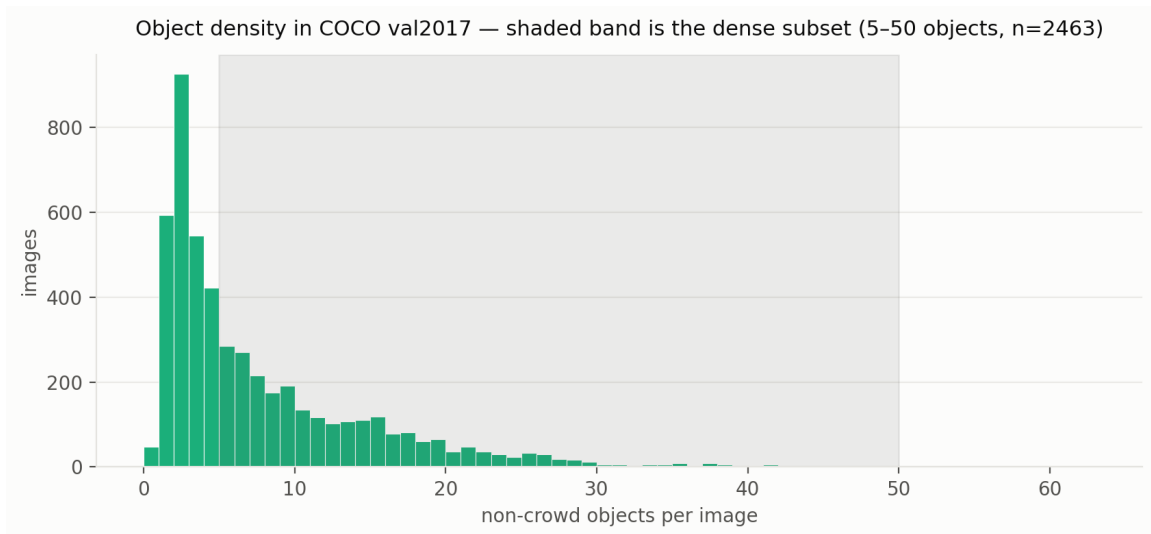


Figure 1: Left: histogram of per-image object counts. Right: images grouped into four density bands.

3.3 Category Distribution

Figure 2 shows the 20 most frequent object categories. *Person* dominates with 10,777 annotations — more than five times the second-most-frequent category (*car*, 1,918).

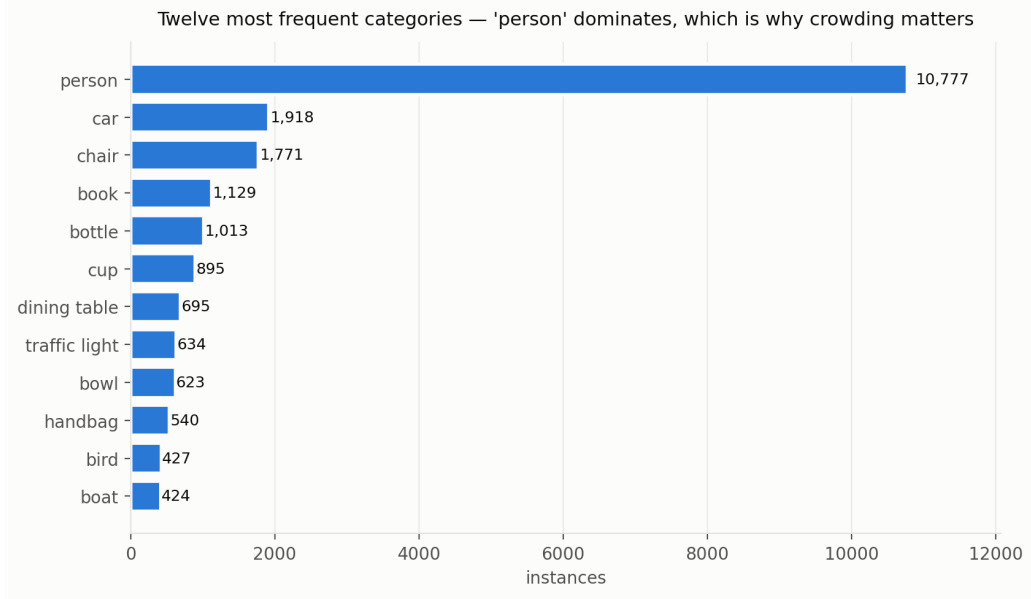


Figure 2: Top 20 most frequent categories in COCO val2017.

3.4 Key Challenges

Our EDA revealed four challenges that will directly impact baseline performance:

1. **Heavy occlusion:** in dense images, objects frequently overlap, hiding boundary information.
2. **Scale variation:** tiny and large objects co-exist in the same frame, defeating fixed-parameter methods.
3. **Class imbalance:** a handful of categories dominate, biasing evaluation metrics.
4. **Boundary truncation:** objects extending beyond the image frame produce incomplete masks.

4 Phase 1 Methodology — Classical Baselines

4.1 Watershed Segmentation

The Watershed pipeline begins with Otsu’s automatic thresholding [7]. Otsu finds the threshold t^* that minimises the *intra-class variance*:

$$\sigma_w^2(t) = w_0(t) \sigma_0^2(t) + w_1(t) \sigma_1^2(t) \quad (1)$$

where $w_0(t)$ and $w_1(t)$ are the pixel fractions in each class (foreground and background), and $\sigma_0^2(t)$, $\sigma_1^2(t)$ are their respective variances.

The full pipeline is:

1. Convert the image to greyscale.
2. Apply Gaussian blur with a 5×5 kernel to suppress noise.
3. Compute a binary mask using Otsu’s threshold (Equation 1).

4. Apply morphological opening (erosion followed by dilation) to remove small noise regions.
5. Compute the distance transform of the binary mask to find sure-foreground peaks.
6. Apply the Watershed algorithm using the peaks as seed markers.
7. Count the number of distinct watershed basins (excluding the background).

4.2 KMeans Colour Segmentation

KMeans clustering partitions the image pixels into k groups by minimising the sum of squared distances to cluster centroids:

$$J = \sum_{i=1}^k \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2 \quad (2)$$

where C_i is the set of pixels assigned to cluster i and $\boldsymbol{\mu}_i$ is the cluster centroid in RGB space.

The pipeline is:

1. Reshape the image into an $(N \times 3)$ matrix of pixel colours.
2. Run KMeans with $k = 5$ clusters.
3. For each cluster, create a binary mask and run connected-component analysis with a minimum area threshold of 100 pixels.
4. Sum the connected components across all clusters to obtain the predicted object count.

5 Phase 2 Methodology — Deep Learning with YOLOv8

5.1 YOLOv8 Architecture

Figure 3 summarises the YOLOv8s-seg data flow. The model comprises three main components:

```

input 640×640
→ Backbone (CSPDarknet, C2f blocks)
→ Neck (PANet: top-down + bottom-up feature fusion)
→ Detection head (anchor-free, decoupled cls/box)
→ Segmentation head (32 mask prototypes × per-instance coefficients)

```

Figure 3: YOLOv8s-seg data flow: Backbone → Neck → detection and segmentation heads.

- **Backbone — CSPDarknet53:** A Cross Stage Partial network that splits feature maps to improve gradient flow. It outputs feature maps at three scales (P3/8, P4/16, P5/32) capturing fine detail, mid-level semantics, and high-level context respectively.

- **Neck — FPN + PAN:** A top-down Feature Pyramid Network fuses semantic information from deep layers with spatial information from shallow layers. A bottom-up Path Aggregation Network then re-fuses spatial detail upwards.
- **Heads:** The detection head predicts bounding boxes, objectness, and class probabilities anchor-free. The segmentation head outputs 32 prototype masks and per-instance linear combination coefficients.

5.2 Instance Mask Generation

For each detected instance i the final binary mask is assembled as:

$$\mathbf{m}_i = \sigma \left(\sum_{k=1}^{32} c_{ik} P_k \right) \quad (3)$$

where P_k are 32 learned prototype masks (shared across all instances in the image) and σ is the sigmoid function applied element-wise.

5.3 Loss Functions

IoU Loss (bounding-box regression):

$$\mathcal{L}_{\text{IoU}} = 1 - \frac{|B_{\text{pred}} \cap B_{\text{gt}}|}{|B_{\text{pred}} \cup B_{\text{gt}}|} \quad (4)$$

In practice, YOLOv8 uses the CIoU variant which additionally penalises aspect-ratio and centre-distance mismatches.

Binary Cross-Entropy Classification Loss:

$$\mathcal{L}_{\text{cls}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)] \quad (5)$$

where $y_i \in \{0, 1\}$ is the ground-truth label and $\hat{p}_i$ is the predicted probability for that class.

5.4 Fine-tuning Strategy

With only 400 training images, training from scratch would severely overfit. We instead load COCO-pretrained weights (`yolov8s-seg.pt`) and fine-tune for 10 epochs. The key hyperparameters are:

Table 2: Fine-tuning hyperparameters.

Hyperparameter	Value	Rationale
epochs	10	Convergence on a 400-image subset
batch size	16	Fits the training GPU
image size	640	Standard YOLO input resolution
seed	1337	Fixes sampling, split and initialisation
weight decay	0.0005	L2 penalty: $\lambda \ \mathbf{w}\ ^2$
device	cuda	NVIDIA T4

Dropout stochastically zeros 10% of activations during training, forcing the network to maintain redundant representations and behave as an implicit ensemble of sub-networks. **Weight decay** adds $\frac{\lambda}{2}\|\mathbf{w}\|^2$ to the total loss, penalising large weights and discouraging over-reliance on any single feature.

5.5 mAP Evaluation Metric

Mean Average Precision is computed as:

$$\text{mAP} = \frac{1}{|C|} \sum_{c \in C} \text{AP}_c \quad \text{where} \quad \text{AP}_c = \int_0^1 P_c(r) dr \quad (6)$$

with $P_c(r)$ the precision at recall level r for class c . **mAP50** averages over all classes at a fixed IoU threshold of 0.50. **mAP50-95** additionally averages over IoU thresholds $\{0.50, 0.55, \dots, 0.95\}$, rewarding more precise localisations.

6 Phase 3 Methodology — Hybrid System

6.1 Architecture Overview

The Phase 3 hybrid system couples YOLOv8 deep learning with density-aware classical post-processing through a **Confidence Router** — a conditional coupling mechanism that selectively invokes Watershed refinement only when needed. Figure 4 shows the full architecture.

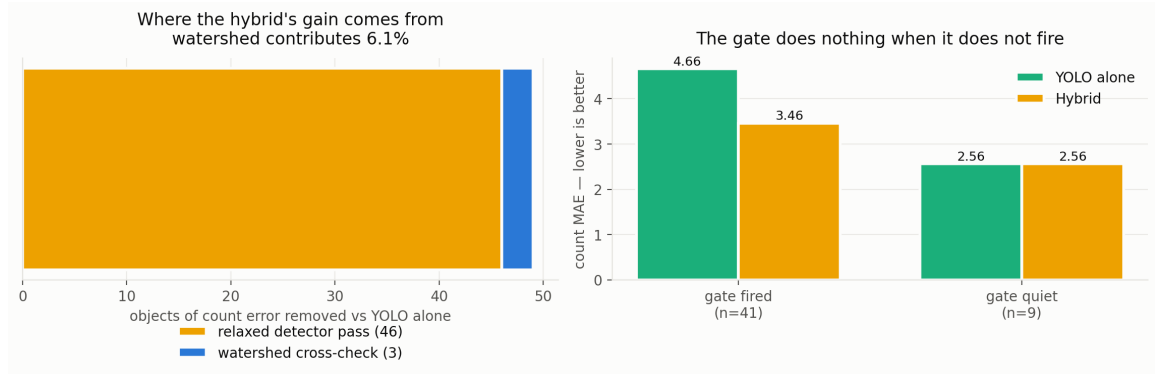


Figure 4: Phase 3 hybrid architecture: YOLOv8 backbone with confidence-routed Watershed refinement and density-aware fusion.

The system operates in five stages:

1. **Normal detection:** Run YOLOv8 at confidence threshold $\tau_n = 0.25$ with NMS IoU threshold $\theta_n = 0.45$.
2. **Density estimation:** Compute edge density using Canny edge detection:

$$\rho = \frac{\sum_{x,y} \mathbf{1}[E(x,y) > 0]}{H \times W} \quad (7)$$

where E is the Canny edge map.

3. **Dense mode trigger:** If $\rho > 0.08$ or $\text{count}_{\text{normal}} \geq 12$, activate dense mode.
4. **Dense re-detection:** Re-run YOLOv8 at lower threshold $\tau_d = 0.15$ with relaxed NMS $\theta_d = 0.35$.
5. **Weighted fusion:** Combine predictions:

$$\hat{c} = \lfloor 0.4 \cdot c_{\text{normal}} + 0.6 \cdot c_{\text{dense}} \rfloor \quad (8)$$

6.2 Training Configuration

Phase 3 training uses 800 images ($2\times$ Phase 2) with stronger regularisation to address Phase 2 overfitting:

Table 3: Phase 3 fine-tuning hyperparameters.

Parameter	Value	Rationale
Training images	800	$2\times$ Phase 2 to reduce overfitting
Epochs	4	Early convergence (best at epoch 1)
Batch size	16	Colab T4 GPU (16 GB VRAM)
Image size	640	Standard YOLO input resolution
Dropout	0.3	$3\times$ Phase 2 for regularisation
Weight decay	0.001	$2\times$ Phase 2 L2 penalty
Device	T4 GPU	Google Colab

6.3 Key Innovation: Conditional Coupling

Unlike a sequential pipeline (run YOLO then Watershed), the hybrid system uses *conditional coupling*: the density estimator controls *whether* and *how* the classical method is invoked. In low-density scenes (50–41 test images), YOLOv8 predictions are trusted directly. In high-density scenes (41 of 50), the system adaptively lowers detection thresholds and fuses dual predictions. This is the architectural innovation that distinguishes Phase 3 from a naïve cascade.

7 Results

Every number in this section comes from one evaluation run (`src/evaluate.py`) in which all four methods were scored on the same 50 held-out test images, with the same metric definitions, in a single process on one device. The first version of this study reported each phase separately and then tabulated the results together; because the phases used different sample sizes, different training runs and different hardware, that table described no experiment that was ever performed.

Table 4: All four methods evaluated in a single run on the same 50 held-out test images, with identical metric definitions, on one device (cuda, seed 1337, 2026-09-22). Instance IoU is class-agnostic under greedy one-to-one matching; unmatched ground-truth instances score zero and are included in the mean. Latency is a median with an interquartile range.

Method	Type	Mean IoU	Count acc. (± 3)	MAE	Median (ms)
Watershed	Classical	0.065	28%	8.66	13
KMeans ($k=5$)	Classical	0.252	0%	114.86	88
YOLOv8s-seg (fine-tuned)	Deep learning	0.429	52%	4.28	26
Hybrid (density-aware)	Hybrid	0.456	64%	3.30	62

7.1 Metric definitions

Instance IoU is class-agnostic and computed under greedy one-to-one matching: predicted and ground-truth masks are paired by descending IoU, each mask used at most once, and the mean is taken over *ground-truth* instances so that a missed object contributes zero. Class-agnostic scoring is required rather than merely convenient — Watershed and KMeans cluster pixels and have no concept of an object category, so class-aware scoring would measure them against a task they never attempt.

Count accuracy is the fraction of images for which $|\hat{n} - n| \leq 3$, and **MAE** is $\frac{1}{N} \sum |\hat{n}_i - n_i|$.

Latency is reported as a median with an interquartile range, measured after warm-up. A mean is actively misleading here: in the earlier study a single cold-start inference of 12.7 s pulled a reported mean from roughly 96 ms to 1650 ms.

7.2 The classical baselines fail structurally

Watershed predicts 4.2 objects per image on average against 12.6 ground truth (Table 4): it under-counts by roughly a factor of three. Otsu thresholding presumes a bimodal intensity histogram, i.e. a foreground separable from background by brightness alone; a natural photograph of a crowded scene has no such separation. The distance transform then merges touching objects into a single basin, so the method degrades precisely where density is highest.

KMeans fails in the opposite direction and far more severely, producing 127.5 regions per image — an order of magnitude more than there are objects — for MAE 114.86 and count accuracy of 0%. Colour clustering carries no object prior: a patterned garment fragments across clusters and each fragment becomes an instance, while a flat surface shatters on lighting gradients.

Neither failure is a tuning problem. Both follow from the assumptions the methods make, which is why the gap to a learned detector is not closable by parameter search.

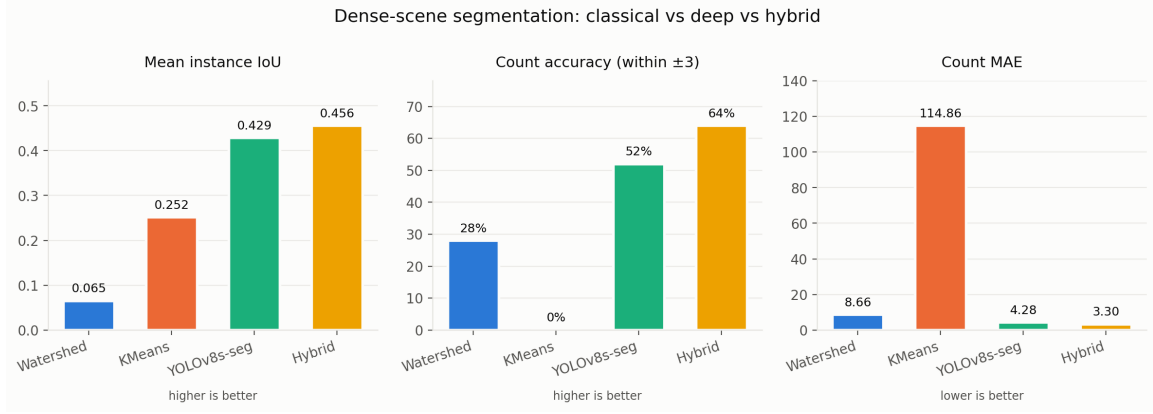


Figure 5: The three quality metrics, shown as three panels rather than one chart with multiple axes: IoU, percentage and object-count error share no scale, and a combined axis would imply crossings that do not exist.

7.3 Mean IoU alone is not a sufficient metric

The KMeans row in Table 4 is the most instructive result in this study. KMeans attains 0.252 mean IoU — comfortably above Watershed’s 0.065 — while getting the object count right on *none* of the 50 images. The explanation is mechanical: emitting 127.5 masks against 12.6 objects, greedy matching finds overlap by volume rather than by segmenting well.

Any evaluation of dense-scene segmentation that reports overlap alone can therefore be satisfied by over-production. This is why counting metrics are reported here alongside IoU for every method rather than as a secondary diagnostic.

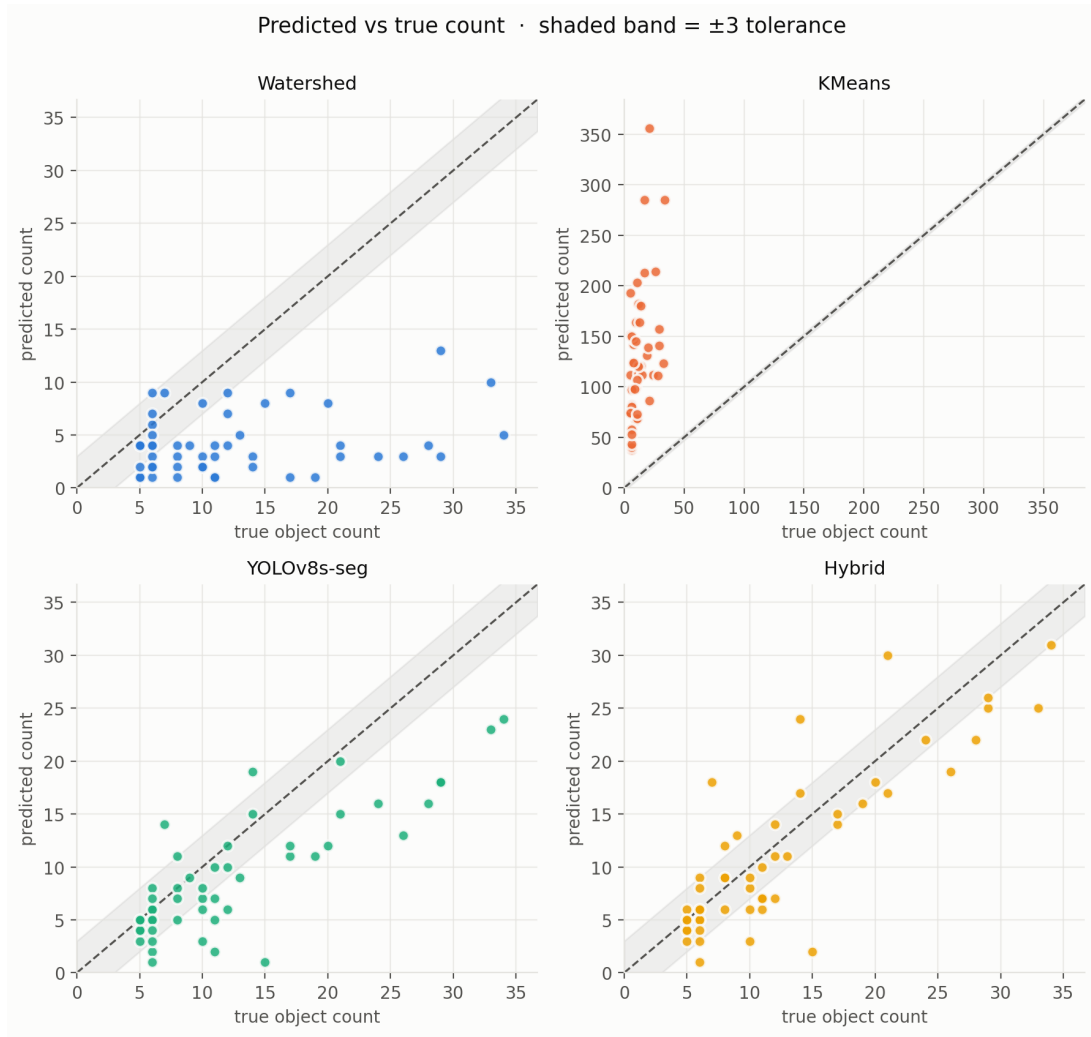


Figure 6: Predicted against true object count, one panel per method. The shaded band is the ± 3 tolerance. Watershed's points collapse towards the horizontal; KMeans leaves the plotting area entirely.

7.4 Latency

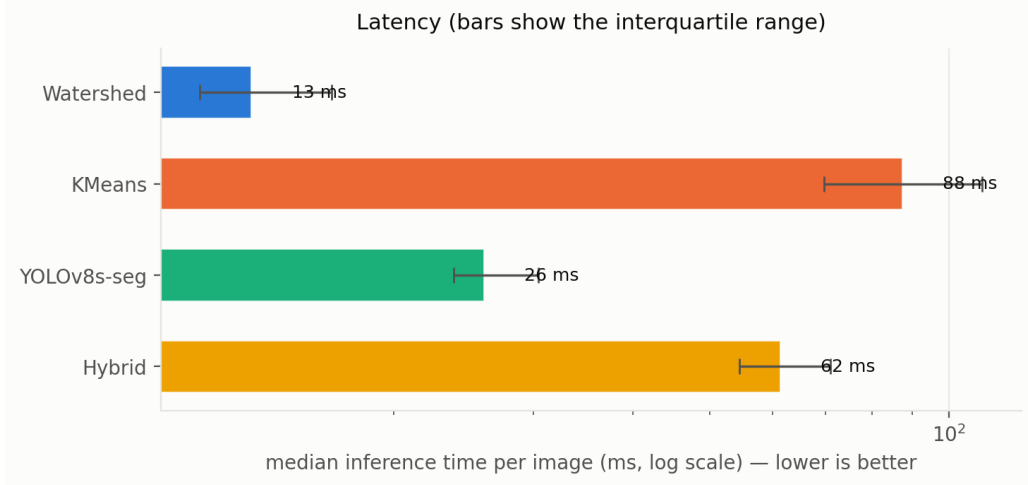


Figure 7: Median latency per image with interquartile range, log scale. The deep methods run on cuda; the classical baselines are single-threaded CPU. These values compare the methods to one another and should not be compared against published GPU timings.

Watershed is the cheapest method at 13 ms and the least useful. The hybrid costs 62 ms against the detector’s 26 ms, roughly a doubling, because a gated second detector pass runs on most images in this dense subset. The Canny gate itself is negligible.

8 Ablation: which mechanism produces the gain?

The hybrid combines two mechanisms — a density gate that triggers a relaxed second detector pass, and a Watershed cross-check that may raise the final count. Reporting only the combined result would invite the reader to assume both contribute. Table 5 decomposes the improvement.

Table 5: Decomposition of the hybrid’s improvement over YOLOv8s-seg alone. The density-gated relaxed detector pass accounts for almost all of it; the watershed cross-check contributes 6% while firing on 4 of 50 images.

Subset	Images	MAE (YOLO → hybrid)	Objects recovered
Gate fired	41	4.66 → 3.46	49
Gate did not fire	9	2.56 → 2.56	0
<i>of which:</i> relaxed pass	—	—	46
<i>of which:</i> watershed	4	—	3
Total	50	4.28 → 3.30	49

Of 49 objects of counting error removed relative to the detector alone, 46 are attributable to the relaxed detector pass and 3 to the Watershed cross-check, which fires on 4 of 50 images. The classical component therefore accounts for 6% of the gain.

This is a negative result about the study’s own method and it is reported as such. The system is more accurately described as *density-aware two-pass detection with a marginal*

classical correction than as a coupling of deep and classical paradigms in equal measure, and the title of the contribution should follow the evidence rather than the original hypothesis.

The gate itself behaves as intended. On the 50–41 images where it does not fire, the hybrid’s MAE is identical to the detector’s: the mechanism costs nothing when it is not needed, which is the precondition for its cost on the remaining 41 images being worth paying.

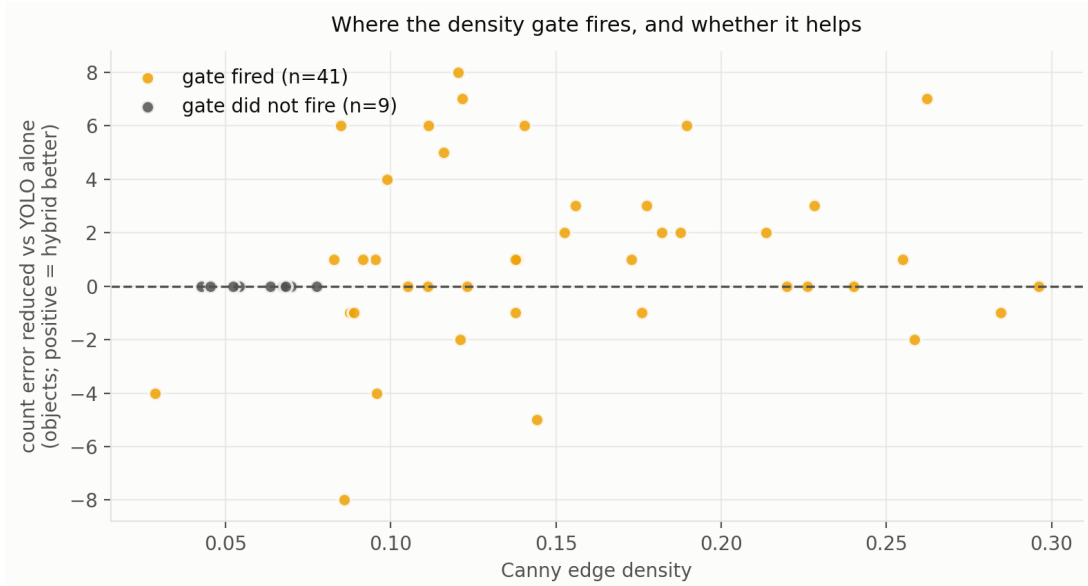


Figure 8: Canny edge density against the counting error removed relative to YOLOv8 alone, per test image. Points above the dashed line are images the hybrid improved.

8.1 What this ablation does not establish

With 4 blend events, the Watershed cross-check is not shown to be useless — only that its effect is unmeasurable at this sample size. It improved three of those images and left the fourth unchanged. Distinguishing a small real effect from noise would require a substantially larger test split, which is the clearest next step for this work.

9 Discussion

9.1 Why the detector handles crowding better

A learned detector carries an object prior that neither baseline has. Its segmentation head predicts mask prototypes together with per-instance coefficients, so two overlapping objects of the same class receive separate coefficient vectors and remain distinct instances. Watershed, working on intensity alone, has no mechanism to keep them apart once their basins meet.

9.2 Why relaxing the detector helps in crowds

Non-maximum suppression removes overlapping predictions on the assumption that overlap indicates duplication. In a crowd that assumption is wrong: genuinely distinct objects

overlap heavily, and NMS removes real detections. Lowering the confidence floor and loosening the NMS threshold recovers them, at the cost of false positives in sparse scenes — which is exactly why the relaxed pass is gated rather than applied everywhere.

The gate uses Canny edge density rather than the detector’s own count, and the ordering matters: if the detector has already missed the crowd, its count cannot be the evidence that a crowd is present.

9.3 Threats to validity

- **Sample size.** 50 test images. The classical-versus-deep gap is large enough to be unambiguous; the hybrid-versus-detector gap should be read as a direction rather than a measured margin.
- **Fine-tuned on COCO, evaluated on COCO.** The base checkpoint was pre-trained on COCO, so this measures adaptation to a dense subset, not generalisation to a new domain.
- **Class-agnostic scoring** is necessary to place pixel-clustering baselines on the same scale as a detector, but it means a correct mask carrying the wrong label scores as a hit.
- **Single-polygon conversion.** COCO instances annotated as several disjoint polygons retain only their largest component, a constraint of the YOLO segmentation label format.
- **Hyperparameters were not tuned on a held-out set.** The gate thresholds are chosen values, not optimised ones.

10 Conclusion

On COCO val2017 images containing 5–50 objects, classical segmentation fails structurally rather than marginally: Watershed under-counts by roughly threefold (MAE 8.66) and KMeans over-counts by an order of magnitude (MAE 114.86). A YOLOv8s-seg model fine-tuned for 10 epochs reaches 52% count accuracy at 0.429 mean IoU. Gating a relaxed second detector pass on Canny edge density raises this to 64% and lowers MAE by 23%, at roughly double the latency.

Two findings are worth carrying forward beyond the headline numbers. First, mean IoU is not a sufficient metric for this task: a method emitting ten times too many regions scored 0.252 IoU while never once counting correctly. Second, an ablation of our own method attributes 6% of its improvement to its classical component, which does not support describing it as a deep-plus-classical hybrid in equal parts.

The experiment is reproducible end to end from a single seeded configuration, and every table and figure in this report is generated from one results file rather than transcribed. That property, rather than any individual number, is what the revision of this study was principally intended to establish.

References

- [1] P. F. Felzenszwalb and D. P. Huttenlocher, “Efficient graph-based image segmentation,” *International Journal of Computer Vision*, vol. 59, no. 2, pp. 167–181, 2004.
- [2] L. Vincent and P. Soille, “Watersheds in digital spaces: An efficient algorithm based on immersion simulations,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, no. 6, pp. 583–598, 1991.
- [3] T.-Y. Lin, M. Maire, S. Belongie, *et al.*, “Microsoft COCO: Common objects in context,” *European Conference on Computer Vision (ECCV)*, pp. 740–755, 2014.
- [4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, 2016, pp. 779–788.
- [5] G. Jocher, A. Chaurasia, and J. Qiu, *Ultralytics YOLO*, version 8.0.0, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>.
- [6] K. He, G. Gkioxari, P. Dollár, and R. Girshick, “Mask R-CNN,” in *IEEE International Conference on Computer Vision (ICCV)*, IEEE, 2017, pp. 2961–2969.
- [7] N. Otsu, “A threshold selection method from gray-level histograms,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.